

FROM MACHINE LEARNING TO LARGE LANGUAGE MODELS

A visual, math-first learning guide for an aspiring AI engineer

What this is A self-contained course, cheat sheet, and practical roadmap. Read it in order the first time, then use the model cards and summaries as a reference.

Track	You will learn
Foundations	Data, features, labels, probability intuition, train/validation/test splits
Classical ML	Regression, classification, trees, ensembles, clustering, PCA
NLP	Bag-of-words, TF-IDF, embeddings, tokenization
Deep learning	Neural networks, loss, backpropagation, optimization, regularization
Transformers & LLMs	Attention, pretraining, decoding, fine-tuning, RAG, evaluation
AI engineering	Pipelines, serving, monitoring, safety, cost, latency, projects

Prepared as a personalized study handbook • October 2026

How to use this course

- Pass 1, intuition: read the “mental model” and study each figure before the formulas.
- Pass 2, mechanics: reproduce the worked examples by hand, then run the code snippets.
- Pass 3, engineering: build the mini-project for each phase and explain trade-offs aloud.
- Use active recall: close the guide and answer the checkpoint questions without notes.
- Aim for understanding, not memorization. You should be able to say what a model optimizes, what assumptions it makes, and how it can fail.

Recommended pace 12 weeks at 5 to 7 hours per week. If you already code comfortably in Python, spend less time on syntax and more time on experiments, error analysis, and deployment.

Course map

Module	Topic	Outcome
0	Prerequisites and ML vocabulary	Read datasets and formulate problems
1	Learning paradigms	Choose supervised, unsupervised, or self-supervised
2	End-to-end workflow	Avoid leakage and build valid experiments
3	Regression	Predict continuous values
4	Classification	Predict classes and probabilities
5	Trees and ensembles	Model nonlinear tabular data
6	Unsupervised learning	Cluster and reduce dimensions
7	Classical NLP	Convert text into useful numeric features
8	Neural networks	Understand forward pass, loss, backprop, gradients
9	Deep learning architectures	Recognize MLPs, CNNs, RNNs, embeddings
10	Transformers	Compute and interpret self-attention
11	LLMs	Understand pretraining, decoding, adaptation, evaluation
12	AI engineering	Build reliable, observable AI systems
13	Roadmap & projects	Turn concepts into a portfolio

Module 0 — Prerequisites and the language of ML

Core mental model A model is a parameterized function. Training searches for parameter values that make its predictions useful on examples it has not seen.

0.1 The objects you manipulate

Term	Meaning	Example
Sample / observation	One row or case	One house
Feature x	Input variable used to predict	square feet
Target y / label	Desired output	sale price

Parameter θ	Value learned during training	regression slope
Hyperparameter	Chosen outside training	tree depth, learning rate
Prediction $\hat{y}$	Model output	predicted price
Loss L	Penalty for wrong prediction	squared error
Metric	How humans judge performance	RMSE, F1, latency

0.2 Minimum math toolkit

- Algebra: rearranging equations, exponents, logarithms, summations.
- Linear algebra: vectors, matrices, dot products, matrix multiplication, shape tracking.
- Calculus: derivative as slope; partial derivative as sensitivity to one parameter; chain rule.
- Probability: conditional probability, expectation, variance, distributions, Bayes rule.
- Statistics: sampling, correlation versus causation, uncertainty, confidence intervals.

Dot product: $\mathbf{x} \cdot \mathbf{w} = \sum_i x_i w_i$

Shape habit Write tensor shapes next to every operation. If X has shape [batch, features] and W has shape [features, outputs], then XW has shape [batch, outputs].

0.3 A tiny worked example

1. Input features are $\mathbf{x} = [2 \text{ bedrooms}, 1.5 \text{ thousand sq ft}]$.
2. Weights are $\mathbf{w} = [\$30\text{k per bedroom}, \$120\text{k per thousand sq ft}]$, with bias $b = \$50\text{k}$.
3. Compute dot product: $2 \times 30 + 1.5 \times 120 = 60 + 180 = 240$.
4. Add bias: $\hat{y} = 240 + 50 = \$290\text{k}$.
5. If true price is $\$310\text{k}$, residual $y - \hat{y} = \$20\text{k}$. Squared error is $20^2 = 400$ in $(\$k)^2$.

Module 1 — Supervised, unsupervised, and other learning paradigms

Paradigm	Data signal	Goal	Examples
Supervised	Inputs X plus labels y	Learn $X \rightarrow y$	price prediction, spam detection
Unsupervised	Inputs X only	Find structure	K-means, PCA, anomaly detection
Self-supervised	Labels created from data itself	Learn representations	predict masked/next tokens
Semi-supervised	Few labels + many unlabeled examples	Use both efficiently	pseudo-labeling
Reinforcement learning	Rewards from actions and environment	Learn a policy	robot control, preference optimization

1.1 Supervised learning

- Regression outputs a continuous number: cost, temperature, demand.
- Classification outputs a class or class probability: fraud/not fraud, category A/B/C.
- Training examples teach the mapping; validation chooses settings; test estimates final generalization.

1.2 Unsupervised learning

- Clustering groups similar examples, but a cluster is not automatically a meaningful real-world class.
- Dimensionality reduction compresses information into fewer coordinates for visualization, noise reduction, or downstream modeling.
- Anomaly detection identifies points unlike normal data and often requires careful thresholding.

1.3 The decision checklist

6. Define the decision the system must support.
7. Identify the prediction unit: one customer, document, image, request, or time window.
8. Ask whether a trustworthy target exists.
9. Specify what information is available at prediction time.
10. Choose a metric that matches the cost of errors.
11. Build a simple baseline before a complex model.

Common mistake Choosing an algorithm before defining the target and evaluation metric. Problem formulation usually matters more than model novelty.

Module 2 — The end-to-end machine learning workflow

2.1 The loop

Step	Key question	Deliverable
1. Frame	What decision and metric matter?	problem statement
2. Collect	Is data representative and permitted?	versioned dataset
3. Explore	What is missing, skewed, duplicated, or leaked?	EDA report
4. Split	How will future data differ?	train/validation/test sets
5. Baseline	What does a simple rule achieve?	reference score
6. Train	Which pipeline and model?	reproducible experiment
7. Evaluate	Where and why does it fail?	error slices
8. Deploy	How is inference served?	API/batch job
9. Monitor	Has data or quality drifted?	alerts and dashboards

2.2 Splits and leakage

- Random split works when examples are independent and identically distributed.
- Time split is safer for forecasting or systems where the future must never influence the past.
- Group split keeps related entities, such as users or patients, entirely in one split.
- Data leakage occurs when training sees information unavailable at prediction time, including preprocessing fitted on all data.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train, y_train) # scaler fits only on training data
```

Model complexity and generalization

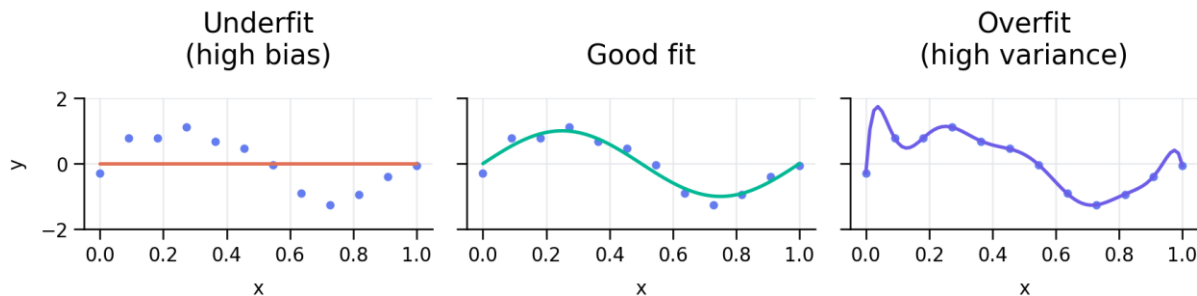


Figure 1. Underfitting, a useful fit, and overfitting.

Use validation data to detect overfitting

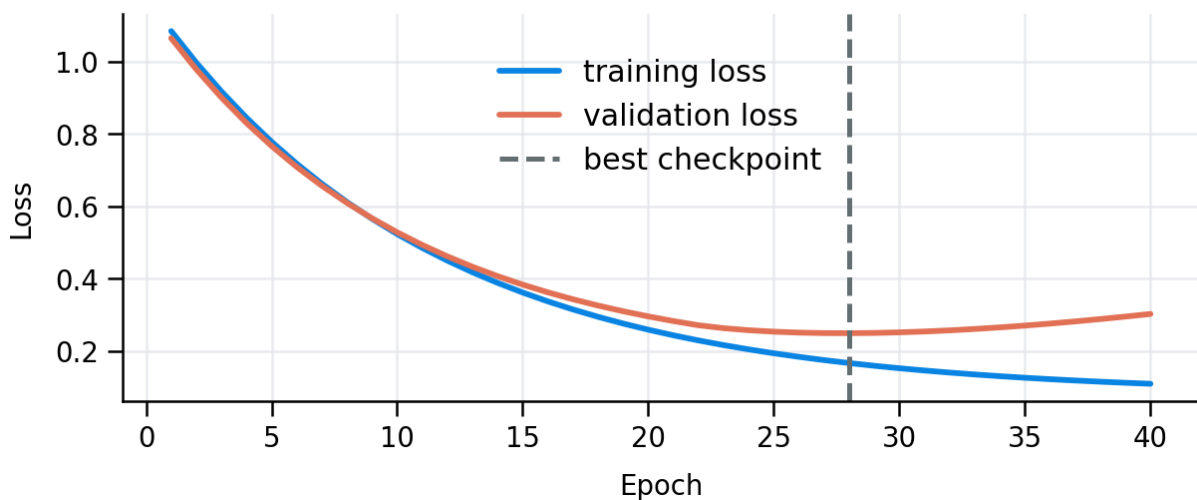


Figure 2. Training and validation curves reveal when optimization stops improving generalization.

Module 3 — Regression models

3.1 Linear regression

Mental model Fit the flat plane that best explains the target as a weighted sum of features.

$$\text{Prediction: } \hat{y} = w_1x_1 + w_2x_2 + \dots + w_px_p + b$$

$$\text{Mean squared error: } \text{MSE} = (1/n) \sum_i (y_i - \hat{y}_i)^2$$

Linear regression minimizes prediction residuals

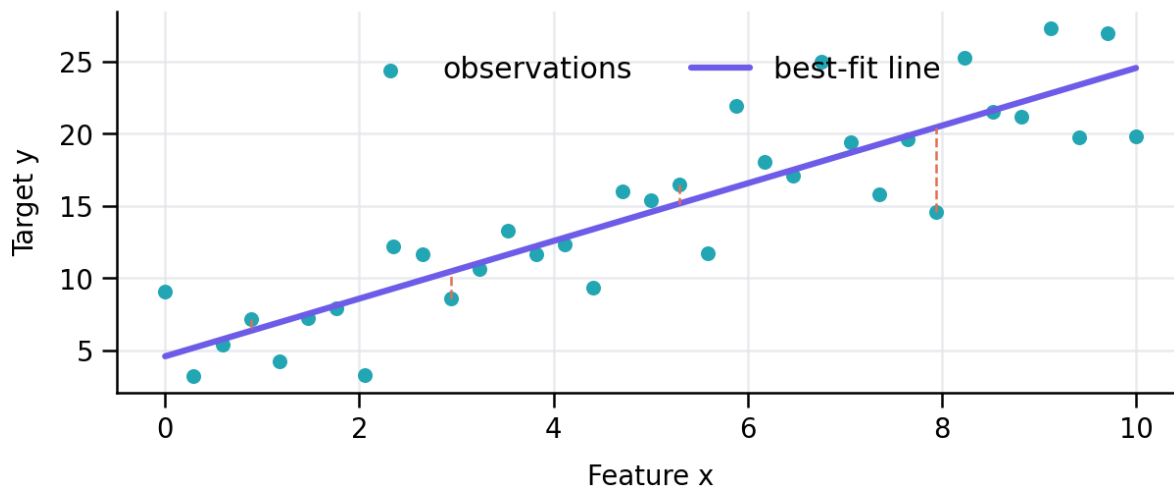


Figure 3. Residuals are the vertical gaps between predictions and observations.

Worked gradient step for one parameter

12. Use model $\hat{y} = wx$ with $x = 2$, true $y = 7$, starting $w = 1$.
13. Prediction: $\hat{y} = 1 \times 2 = 2$.
14. Loss: $L = (y - \hat{y})^2 = (7 - 2)^2 = 25$.
15. Derivative: $dL/dw = 2(\hat{y} - y)x = 2(2 - 7) \times 2 = -20$.
16. With learning rate $\eta = 0.1$, update $w \leftarrow w - \eta(dL/dw) = 1 - 0.1(-20) = 3$.
17. New prediction is $3 \times 2 = 6$, much closer to 7.

3.2 Regularization

Method	Penalty	Effect
Ridge / L2	$\lambda \sum w^2$	shrinks weights smoothly; handles correlated features
Lasso / L1	$\lambda \sum w $	can drive some weights to zero; feature selection
Elastic Net	L1 + L2	mixes sparsity and stable shrinkage

3.3 Regression metrics

Metric	Formula / intuition	Use when
MAE	average error	every unit of error has similar cost
RMSE	square root of average squared error	large errors deserve extra penalty
R ²	1 - SSE/SST	compare explained variation; can be negative on test data
MAPE	average percent error	scale-relative error; avoid near-zero targets

Module 4 — Classification models and evaluation

4.1 Logistic regression

Mental model Compute a linear score, then squeeze it into a probability from 0 to 1 with the sigmoid function.

$$z = w \cdot x + b \quad \text{and} \quad p(y=1|x) = \sigma(z) = 1 / (1 + e^{-z})$$

$$\text{Binary cross-entropy: } -[y \log(p) + (1-y) \log(1-p)]$$

4.2 K-nearest neighbors (KNN)

K-nearest neighbors: local voting

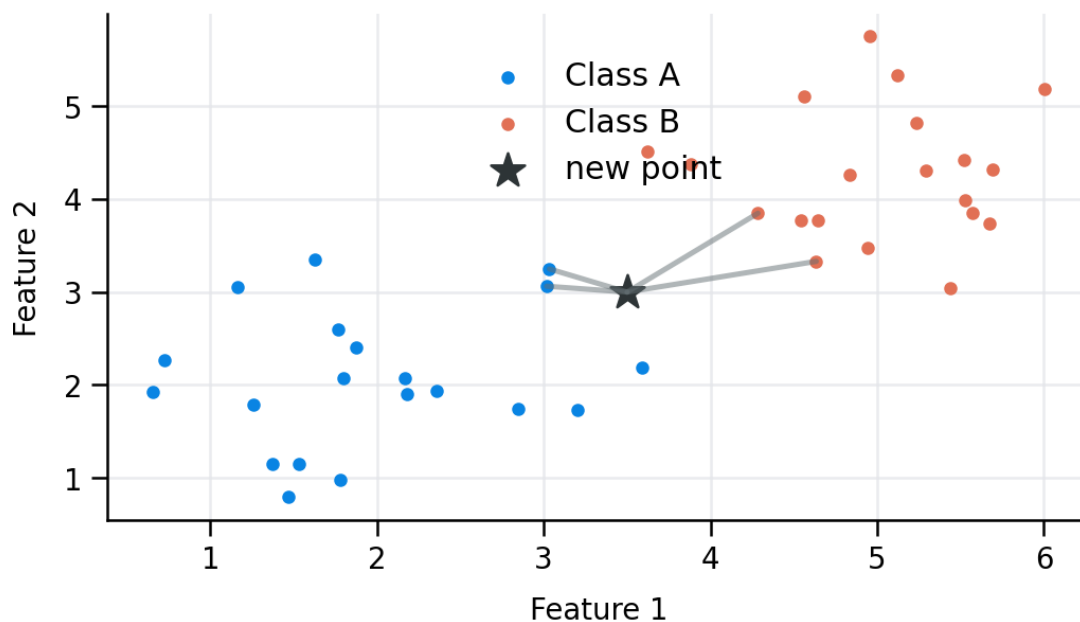


Figure 4. KNN predicts from nearby labeled examples rather than learning a global equation.

- Scale features first because distance is sensitive to units.
- Small K is flexible and noisy; large K is smoother and may blur boundaries.
- Prediction can be slow because the model searches stored training points.

4.3 Naive Bayes and support vector machines

Model	Core idea	Good fit	Watch out
Naive Bayes	combine class prior with feature likelihoods	text and small data baselines	strong independence assumption
SVM	maximize the margin between classes	medium-size, high-dimensional data	scaling and kernel tuning
Linear SVM	linear margin in feature space	sparse TF-IDF text	probabilities require calibration

4.4 Confusion matrix and metrics

Confusion matrix

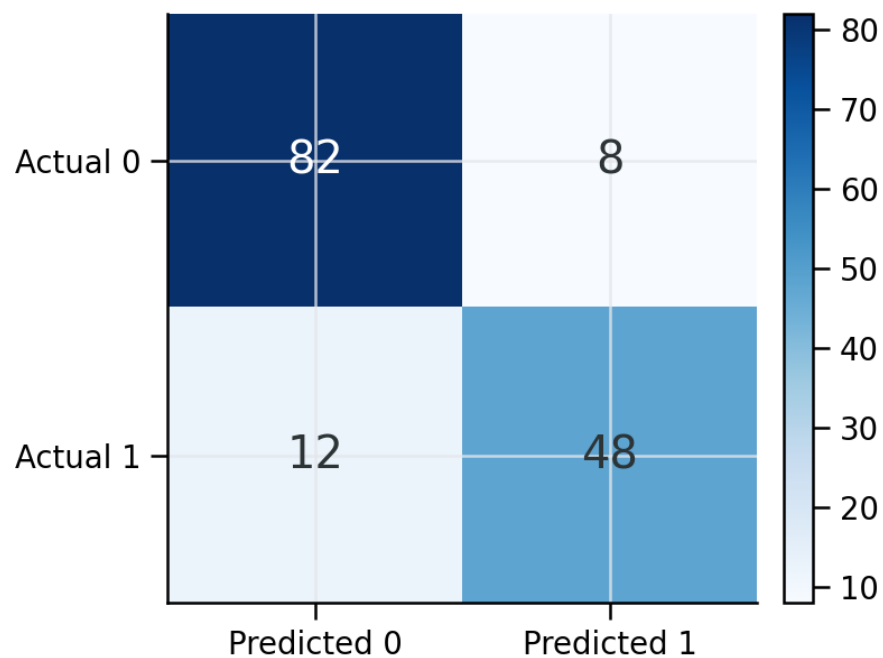


Figure 5. Every binary classification prediction falls into one of four cells.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Thresholds are product decisions The model returns a score or probability; the threshold converts it into an action. Lowering the threshold usually increases recall and false positives. Choose it using real error costs.

Module 5 — Decision trees, random forests, gradient boosting, and XGBoost

5.1 Decision trees

- A tree repeatedly splits examples using rules such as age < 35.
- Each split aims to create purer child nodes. Classification often uses Gini impurity or entropy; regression uses squared-error reduction.
- Trees capture nonlinear interactions and require little scaling, but deep trees overfit.

$$\text{Gini impurity} = 1 - \sum_k p_k^2$$

5.2 Bagging versus boosting

Idea	Training pattern	Primary benefit	Representative model
Bagging	many models independently on resampled data	reduces variance	Random Forest
Boosting	models sequentially correct prior errors	reduces bias; strong accuracy	XGBoost, LightGBM, CatBoost

5.3 Random forest

- Each tree sees a bootstrap sample and a random subset of features.
- Predictions are averaged or voted across trees.
- Useful default for robust tabular baselines and nonlinear relationships.

5.4 Gradient boosting and XGBoost

Mental model Start with a weak prediction. Fit the next small tree to the mistakes or gradients. Add it to the ensemble with a controlled step size. Repeat.

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x})$$

Knob	Effect if increased	Typical risk
n_estimators	more corrective trees	overfitting / slower training
learning_rate η	larger contribution per tree	unstable or overly aggressive fit
max_depth	more interactions	overfitting
subsample	fraction of rows per tree	too low can underfit
colsample	fraction of features	too low can miss signal

regularization	simpler leaf weights	too much can underfit
----------------	----------------------	-----------------------

```
from xgboost import XGBClassifier
```

```
model = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss"
)
model.fit(X_train, y_train)
```

Rule of thumb For structured/tabular data, gradient-boosted trees are often a top baseline. For raw images, audio, and large-scale language, deep learning usually has the stronger inductive bias.

Module 6 — Unsupervised learning

6.1 K-means clustering

K-means: discover groups by minimizing within-cluster distance

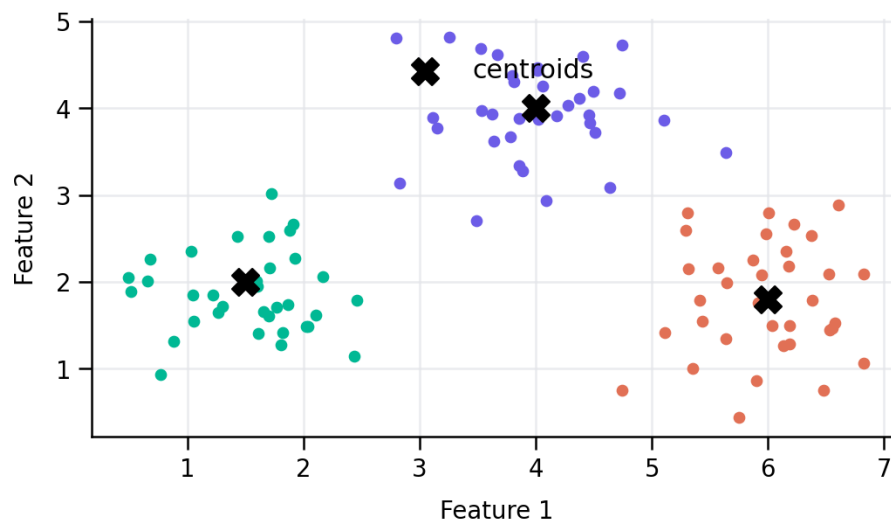


Figure 6. K-means alternates assignment and centroid update steps.

Objective: minimize $\sum_i ||\mathbf{x}_i - \mu_c(i)||^2$

18. Choose K initial centroids.
 19. Assign each point to its nearest centroid.
 20. Replace each centroid with the mean of assigned points.
 21. Repeat until assignments or centroids stabilize.
- Scale features so one unit does not dominate distance.
 - Works best for roughly spherical, similarly sized clusters.
 - Use silhouette score and domain review, not the elbow plot alone.

6.2 Other clustering methods

Method	Strength	Weakness
Hierarchical	dendrogram reveals nested structure	can be costly on large data
DBSCAN	finds irregular shapes and noise; no K required	sensitive to density and scaling
Gaussian mixture	soft probabilistic memberships	assumes mixture distribution; initialization matters

6.3 Principal component analysis (PCA)

Mental model Rotate the coordinate system to directions that capture the most variance, then keep only the leading directions.

- Center and usually scale features.
- The first principal component is the direction of maximum variance.
- Components are linear combinations, not original features.
- PCA can compress and denoise, but variance is not the same as task usefulness.

Module 7 — Classical NLP: from text to vectors

7.1 Text preprocessing

- Tokenization splits text into units: words, subwords, characters, or bytes.
- Normalization may include case folding or Unicode normalization, but aggressive cleaning can remove meaning.
- Stop-word removal and stemming can help classical models; they are not default requirements for transformers.
- Always fit vocabulary and statistics only on training data.

7.2 Bag-of-words and n-grams

Mental model Create one feature per vocabulary item and count how often it occurs. N-grams preserve short local phrases such as “not good.”

Document	machine	learning	is	useful
“machine learning is useful”	1	1	1	1
“machine learning learning”	1	2	0	0

7.3 TF-IDF

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \log(N / \text{DF}(t))$$

22. Suppose there are $N = 100$ documents.
23. The term “gradient” occurs 3 times in document d , so raw $\text{TF} = 3$.
24. It appears in $\text{DF} = 5$ documents total.
25. $\text{IDF} = \log(100/5) = \log(20) \approx 3.0$ using natural log.
26. $\text{TF-IDF} \approx 3 \times 3.0 = 9.0$. A frequent term in one document but rare across the corpus gets a high weight.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

text_model = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_features=50_000),
    LogisticRegression(max_iter=1000)
)
text_model.fit(train_texts, train_labels)
```

7.4 Word embeddings

- Dense vectors place tokens with related usage near one another.
- Word2Vec learns from local context using skip-gram or CBOW objectives.
- Static embeddings assign one vector per word; contextual embeddings assign different vectors depending on the sentence.
- Cosine similarity compares direction rather than vector magnitude.

$$\text{cosine}(a,b) = (a \cdot b) / (||a|| ||b||)$$

7.5 Classical NLP versus transformers

Approach	Advantages	Limitations
TF-IDF + linear model	fast, cheap, interpretable, excellent baseline	sparse; limited context and semantics
Static embeddings	dense semantics; reusable	one vector per word; weak context
Transformer embeddings	contextual and transferable	more compute, latency, and complexity

Module 8 — Neural networks from first principles

Each edge has a weight; each node applies activation

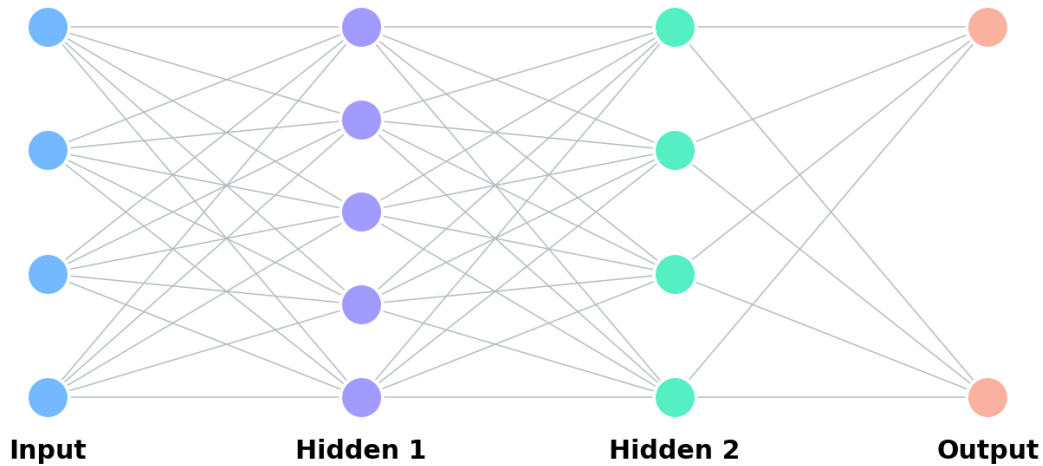


Figure 7. A multilayer perceptron composes linear transformations and nonlinear activations.

8.1 The forward pass

$$\text{Layer: } z = Wx + b \quad \text{Activation: } a = f(z)$$

- Weights mix information across inputs.
- Bias shifts activation thresholds.
- Activation functions introduce nonlinearity; without them, stacked linear layers collapse into one linear transform.

Activation	Definition / behavior	Typical use
ReLU	$\max(0, z)$	hidden layers; simple and efficient
GELU	smoothly gates values	transformers
Sigmoid	0 to 1	binary output probability
Softmax	class scores $\rightarrow$ probabilities summing to 1	multiclass output

8.2 Loss functions

Task	Common loss	What it encourages
Regression	MSE / Huber	numerically close predictions
Binary classification	binary cross-entropy	high probability on correct binary label
Multiclass	categorical cross-entropy	high probability on correct class
Language modeling	token cross-entropy	high probability on the next token

Metric learning	contrastive / triplet	similar examples close; dissimilar apart
-----------------	-----------------------	--

8.3 Gradient descent

Gradient descent follows the slope toward lower loss

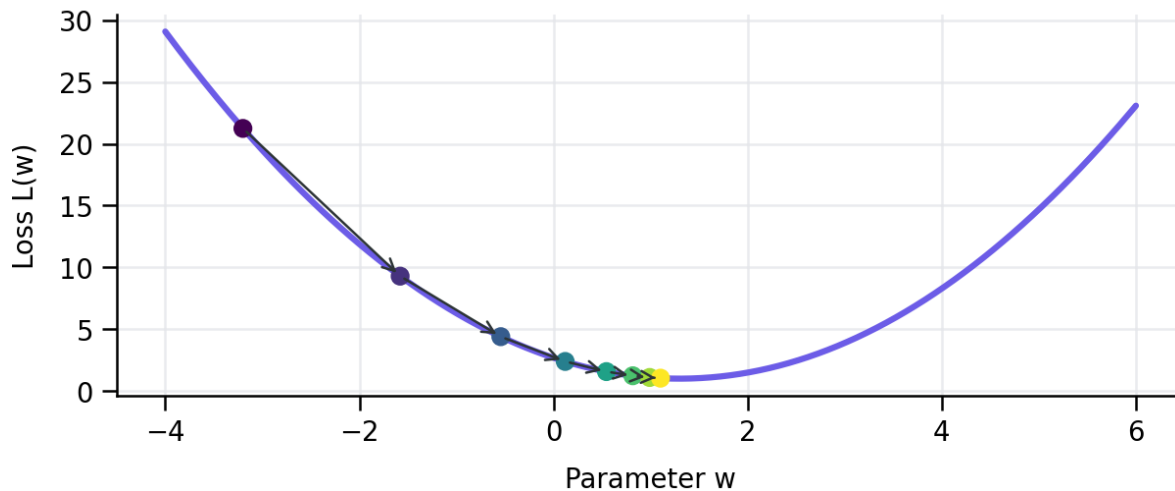


Figure 8. The gradient points uphill; subtracting it moves parameters downhill.

$$\text{Parameter update: } \theta \leftarrow \theta - \eta \nabla_{\theta} L$$

- Learning rate η controls step size. Too high can diverge; too low trains slowly.
- Batch gradient uses all examples, SGD uses one, and mini-batch training uses a practical middle ground.
- Momentum smooths updates; Adam adapts step sizes per parameter.

Module 8 continued — Backpropagation step by step

Core idea Backpropagation is repeated chain rule. It assigns responsibility for the final loss to every upstream parameter.

A one-neuron worked example

27. Inputs: $x_1 = 2$, $x_2 = 1$. Weights: $w_1 = 0.5$, $w_2 = -1$. Bias $b = 0$.
28. Weighted sum: $z = 0.5 \times 2 + (-1) \times 1 + 0 = 0$.
29. Use sigmoid: $\hat{y} = \sigma(0) = 0.5$. True label $y = 1$.
30. With sigmoid plus binary cross-entropy, $\partial L / \partial z = \hat{y} - y = 0.5 - 1 = -0.5$.
31. Gradients: $\partial L / \partial w_1 = (\partial L / \partial z) x_1 = -0.5 \times 2 = -1$; $\partial L / \partial w_2 = -0.5 \times 1 = -0.5$; $\partial L / \partial b = -0.5$.
32. With $\eta = 0.1$: $w_1 \leftarrow 0.5 - 0.1(-1) = 0.6$; $w_2 \leftarrow -1 - 0.1(-0.5) = -0.95$; $b \leftarrow 0.05$.
33. New $z = 0.6 \times 2 - 0.95 + 0.05 = 0.30$, so $\sigma(z) \approx 0.574$. The prediction moved toward $y = 1$.

Computational graph view

Forward	Local derivative	Backward contribution
$z = w_1 x_1 + w_2 x_2 + b$	$\partial z / \partial w_1 = x_1$	$\partial L / \partial w_1 = (\partial L / \partial z) x_1$

$\hat{y} = \sigma(z)$	$\partial\hat{y}/\partial z = \hat{y}(1-\hat{y})$	passes sensitivity through sigmoid
$L = \text{BCE}(y, \hat{y})$	combined result $\partial L/\partial z = \hat{y} - y$	error signal sent backward

```
# Minimal PyTorch training step
optimizer.zero_grad() # clear old gradients
logits = model(x_batch) # forward pass
loss = loss_fn(logits, y_batch)
loss.backward() # backprop computes gradients
optimizer.step() # update parameters
```

8.4 Regularization and stable training

- Weight decay discourages very large weights.
- Dropout randomly removes activations during training.
- Early stopping chooses the best validation checkpoint.
- Batch normalization or layer normalization stabilizes activations.
- Data augmentation creates label-preserving variations.
- Gradient clipping limits exploding gradients.

Module 9 — Deep learning architectures

Architecture	Inductive bias	Common uses
MLP / feed-forward	generic feature interactions	tabular data, final prediction heads
CNN	local patterns and translation structure	images, audio spectrograms, some sequences
RNN / LSTM / GRU	sequential hidden state	time series, legacy NLP, streaming sequences
Transformer	content-based global attention	language, vision, audio, multimodal
Autoencoder	compress then reconstruct	representation learning, denoising
Diffusion model	iterative denoising	image/audio generation

9.1 Convolutional neural networks

- A kernel slides over local regions and reuses the same weights.
- Early layers learn edges and textures; deeper layers compose more abstract patterns.
- Pooling or striding reduces spatial resolution.

9.2 Recurrent networks

$$\mathbf{h}_t = f(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$$

- The hidden state summarizes prior tokens or time steps.

- Vanishing/exploding gradients make long dependencies difficult.
- LSTM and GRU add gates; transformers later improved parallelism and long-range interaction.

9.3 Embeddings as learned lookup tables

- An embedding matrix E has shape [vocabulary size, embedding dimension].
- A token ID selects one row vector. During training, gradients adjust useful token representations.
- Embeddings also represent users, products, categories, graph nodes, and retrieval chunks.

Transfer learning Instead of training from scratch, begin with a model pretrained on a broad task and adapt it. This usually reduces the labeled data and compute required.

Module 10 — Transformers and attention

Why transformers matter They let every token gather information from other relevant tokens using attention, while processing many sequence positions in parallel during training.

10.1 Query, key, and value intuition

- Query Q : what information is this token looking for?
- Key K : what information does each token advertise?
- Value V : what content should be passed if that token is relevant?

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Numerical attention example

34. One query $q = [1, 0]$. Two keys: $k_1 = [1, 0]$, $k_2 = [0, 1]$. Dimension $d_k = 2$.
35. Dot-product scores: $q \cdot k_1 = 1$ and $q \cdot k_2 = 0$.
36. Scale by $\sqrt{2}$: $[1/\sqrt{2}, 0] \approx [0.707, 0]$.
37. Softmax gives approximately $[0.67, 0.33]$.
38. If values are $v_1 = [10, 0]$ and $v_2 = [0, 6]$, output = $0.67v_1 + 0.33v_2 \approx [6.7, 2.0]$.
39. The token receives more information from the first value because its key better matches the query.

Self-attention weights: context-dependent information flow

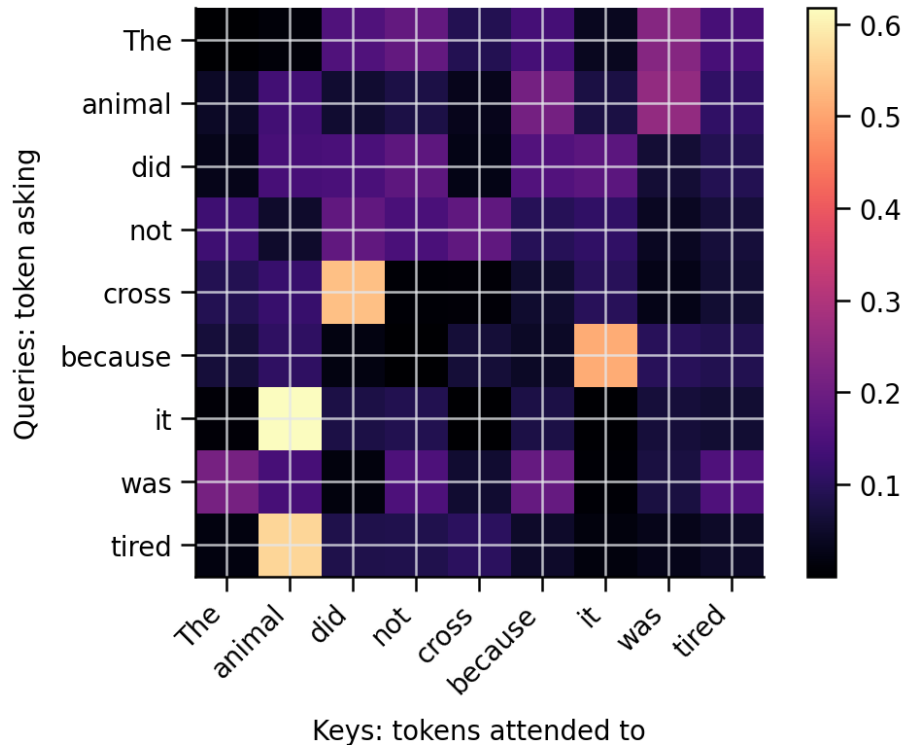


Figure 9. Attention acts like a learned, context-specific weighted lookup.

10.2 Multi-head attention

- Multiple heads use different learned projections of Q, K, and V.
- Heads can specialize in different relationships such as syntax, position, or reference.
- Head outputs are concatenated and projected back into the model dimension.

10.3 A transformer block

Stage	Purpose
Token + position embeddings	represent identity and order
Layer normalization	stabilize activations
Multi-head self-attention	mix information across sequence positions
Residual connection	preserve signal and improve gradient flow
Feed-forward network	transform each position independently
Second residual + normalization	produce block output

10.4 Encoder, decoder, and masking

Family	Attention pattern	Typical use
--------	-------------------	-------------

Encoder-only	bidirectional context	classification, embeddings, extraction
Decoder-only	causal: only prior tokens visible	generation and modern chat LLMs
Encoder-decoder	encoder reads input; decoder generates output	translation, summarization

Module 11 — How large language models work

11.1 Tokenization

- Text is broken into tokens, usually subword or byte-level units.
- Tokens map to integer IDs, then embedding vectors.
- Token count, not character count, drives context usage and inference cost.
- Different tokenizers segment the same text differently.

11.2 Pretraining objective

Autoregressive objective: maximize $\sum_t \log P(\text{token}_t \mid \text{token}_{<t})$

- A decoder-only LLM predicts the next token across massive corpora.
- Cross-entropy compares predicted token probabilities with the actual next token.
- The learned parameters encode statistical structure that supports generation, classification, translation, and reasoning-like behavior.

11.3 Generation and decoding

Method	Behavior	Trade-off
Greedy	always pick highest-probability token	deterministic but can be dull or brittle
Temperature	rescale logits before sampling	lower is focused; higher is diverse
Top-k	sample only among k highest tokens	simple diversity control
Top-p / nucleus	sample smallest set reaching probability p	adaptive candidate set
Beam search	track several high-scoring sequences	useful for constrained generation; costly

Softmax with temperature: $P_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$

11.4 Adaptation methods

Method	What changes	When to use
Prompting	instructions and examples in context	fastest, no training
RAG	retrieve external context at runtime	fresh or private knowledge; citations

Fine-tuning	update many or all weights	special behavior/domain adaptation with enough data
LoRA / PEFT	train small low-rank adapters	lower-cost adaptation
Preference optimization	align responses to human or synthetic preferences	behavior, helpfulness, style

11.5 RAG pipeline

Step	What happens	Failure mode
Ingest	parse and clean sources	missing or corrupted text
Chunk	split documents into retrievable units	too small loses context; too large dilutes relevance
Embed/index	store vectors plus metadata	wrong model, stale index
Retrieve	find relevant chunks	poor query or low recall
Rerank	score candidates more precisely	extra latency
Generate	answer using supplied context	hallucination or weak grounding
Evaluate	measure retrieval and answer quality	only judging fluency

Important distinction RAG changes the model's context, not its weights. Fine-tuning changes behavior stored in weights, but is not a reliable way to inject frequently changing facts.

Module 11 continued — LLM evaluation, limitations, and efficiency

11.6 Evaluation

Layer	Example metrics / checks
Task quality	accuracy, F1, exact match, rubric score
Retrieval	recall@k, MRR, nDCG, context relevance
Grounding	claim support, citation correctness, faithfulness
Safety	harmful-content rate, prompt-injection resistance, privacy
Operations	p50/p95 latency, tokens/sec, error rate, cost/request

Human factors	usefulness, calibration, user correction rate
---------------	---

11.7 Hallucinations and uncertainty

- An LLM optimizes likely token sequences, not guaranteed truth.
- Confident language is not calibrated certainty.
- Mitigate with retrieval, tools, constrained output, verification, abstention, and human review for consequential decisions.
- Evaluate on realistic adversarial and edge cases, not just happy paths.

11.8 Context windows, caching, and inference

- Attention over long sequences is computationally expensive in standard form.
- KV caching stores prior key/value tensors so generation does not recompute the whole prefix each token.
- Batching improves throughput but may increase individual latency.
- Quantization uses fewer bits per weight to reduce memory and often increase speed, with possible quality trade-offs.
- Distillation trains a smaller model to imitate a larger teacher.

11.9 LLM terminology cheat sheet

Term	Plain-English meaning
Parameters	learned numeric weights
Context window	maximum tokens considered for a request
Logits	raw scores before softmax
Perplexity	exponentiated average token loss; lower means better next-token prediction
Embedding	dense numeric representation
Inference	running a trained model
Checkpoint	saved model state
Grounding	connecting output to supplied evidence
Tool calling	model selects a structured external action
Agent	system that plans or loops across model and tool calls

Module 12 — The AI engineer's systems toolkit

Role definition An AI engineer combines model understanding with software engineering, data pipelines, evaluation, deployment, security, and product judgment.

12.1 Production architecture

Layer	Responsibilities
Interface	API, UI, authentication, rate limits
Orchestration	prompt templates, routing, retries, structured outputs, tools
Knowledge	document ingestion, search, vector index, metadata filters
Model	hosted API or self-hosted inference, model versioning
Evaluation	offline test sets, online experiments, human review
Observability	traces, prompts, latency, token usage, errors, quality signals
Governance	privacy, access control, retention, audit, safety policies

12.2 Reliability patterns

- Use schemas and validators instead of parsing free-form prose.
- Set timeouts, bounded retries, circuit breakers, and idempotency for tool calls.
- Separate system instructions, user content, retrieved data, and tool results.
- Treat retrieved text and web content as untrusted input to reduce prompt-injection risk.
- Log enough for debugging while redacting secrets and sensitive data.
- Build fallback behavior for model or retrieval outages.

12.3 ML and LLM observability

Classical ML	LLM applications
feature drift	prompt/input drift
prediction distribution	token and tool-call distribution
ground-truth performance	judge/human/task success
training-serving skew	prompt/template/index version skew
data quality anomalies	retrieval gaps and grounding failures
latency and throughput	latency, tokens/sec, cost, cache hit rate

12.4 Security and responsible delivery

- Apply least privilege to tools, data stores, and service identities.
- Never place secret keys in prompts or logs.

- Use allowlists and validation for generated SQL, code, URLs, and tool arguments.
- Perform threat modeling for prompt injection, data exfiltration, unsafe actions, and supply-chain risks.
- Document intended use, limitations, evaluation coverage, and escalation paths.

Module 13 — A 12-week learning plan

Week	Concepts	Hands-on deliverable
1	problem framing, vectors, probability, splits	EDA notebook + leakage checklist
2	linear/logistic regression, metrics	regression and classification from scratch
3	KNN, trees, random forest	compare decision boundaries and error slices
4	boosting and XGBoost	tabular benchmark with tuning
5	K-means, DBSCAN, PCA	customer or document clustering report
6	TF-IDF, n-grams, embeddings	text classifier baseline
7	neural networks, loss, gradients	NumPy MLP or PyTorch classifier
8	optimization, regularization, CNN/RNN overview	training diagnostic report
9	attention and transformer internals	calculate attention by hand + inspect tensors
10	tokenization, decoding, fine-tuning concepts	small transformer inference notebook
11	RAG, evaluation, tool use	grounded Q&A service with citations
12	deployment, observability, security	FastAPI demo with tests and dashboard

Portfolio project ladder

Level	Project	What makes it stand out
1	Tabular failure-analysis lab	not just accuracy: calibration, slices, drift, explainability
2	Semantic incident triage	cluster logs, classify severity, retrieve similar incidents
3	RAG evaluation workbench	separate retriever, reranker, grounding, latency, cost metrics
4	Adaptive model router	route requests by quality, privacy, latency, and budget
5	Agent reliability sandbox	instrument tool failures, retries, permissions, and prompt injection

How to study each model

40. State the task and output type.
41. Draw the model's input-to-output computation.
42. Write its objective or loss.
43. List assumptions and required preprocessing.
44. Identify key hyperparameters.
45. Choose suitable evaluation metrics.
46. Describe failure modes.
47. Implement a baseline.
48. Perform error analysis and compare against a simpler model.

Master model selection cheat sheet

Situation	Start with	Then consider
Continuous tabular target	linear/ridge regression	random forest, gradient boosting
Binary or multiclass tabular	logistic regression	XGBoost / CatBoost
Sparse text labels	TF-IDF + linear model	fine-tuned transformer
No labels; want groups	K-means	DBSCAN, hierarchical, GMM
High-dimensional visualization	PCA	UMAP/t-SNE for exploration
Image classification	pretrained CNN/vision transformer	fine-tuning
Text generation	pretrained decoder LLM	RAG, PEFT, fine-tuning
Private/fresh knowledge Q&A	retrieval + grounded generation	reranking, hybrid search
Strict low latency	simple model / small encoder	distillation, quantization, caching

Questions to ask before trusting a result

- Was the test set truly unseen and representative?
- Could any feature leak the target or future information?
- Is the metric aligned with business cost and class imbalance?
- How does the model perform across meaningful slices?
- Is it calibrated? Does confidence correspond to observed correctness?
- What happens under missing values, drift, adversarial inputs, and outages?
- Can the pipeline be reproduced from data version, code version, configuration, and random seed?

The formulas worth remembering

Concept	Formula
Linear model	$\hat{y} = w \cdot x + b$

MSE	$(1/n)\sum(y-\hat{y})^2$
Sigmoid	$1/(1+e^{-z})$
Gradient update	$\theta \leftarrow \theta - \eta \nabla \theta L$
Cosine similarity	$(a \cdot b) / (a b)$
TF-IDF	$TF(t,d) \times \log(N/DF(t))$
Self-attention	$\text{softmax}(QK^T/\sqrt{d_k})V$
Precision / recall	$TP/(TP+FP)$, $TP/(TP+FN)$
F1	$2PR/(P+R)$

Knowledge checks and interview prompts

Foundations

- Why can a model with lower training loss have worse test performance?
- When should you use a time split instead of a random split?
- Explain the difference between a parameter and a hyperparameter.
- Give three examples of data leakage.

Classical ML

- Why does KNN need feature scaling while trees usually do not?
- Compare bagging and boosting.
- When would precision matter more than recall, and vice versa?
- Why can accuracy be misleading on an imbalanced dataset?
- What assumptions make K-means fail?

Deep learning

- Why are nonlinear activations necessary?
- Explain the forward pass, loss, backward pass, and optimizer step.
- What does the learning rate control?
- How do residual connections help?
- Why do validation curves matter?

Transformers and LLMs

- Explain Q, K, and V without using equations, then with the attention equation.
- Why does causal masking matter for next-token generation?
- What is the difference between tokenization and embedding?
- When should you choose RAG instead of fine-tuning?
- How would you evaluate a grounded Q&A system end to end?
- What production controls reduce prompt-injection risk?

Teaching test You understand a concept when you can explain it simply, calculate a small example, implement it, and diagnose when it fails.

Recommended references and source notes

- scikit-learn User Guide: supervised learning, unsupervised learning, preprocessing, model selection, evaluation, and common pitfalls. https://scikit-learn.org/stable/user_guide.html
- PyTorch Tutorials, “A Gentle Introduction to torch.autograd”: forward propagation, backward propagation, gradients, and optimizer steps. https://docs.pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html
- Vaswani et al., “Attention Is All You Need,” 2017: the transformer architecture and scaled dot-product attention. <https://arxiv.org/abs/1706.03762>
- Hugging Face LLM Course: NLP tasks, tokenization, transformers, fine-tuning, and the Hugging Face ecosystem. <https://huggingface.co/learn/llm-course>
- James, Witten, Hastie, Tibshirani, An Introduction to Statistical Learning: accessible statistical learning foundations.
- Goodfellow, Bengio, Courville, Deep Learning: neural network fundamentals and optimization.
- Murphy, Probabilistic Machine Learning: a modern, rigorous reference for probabilistic ML.

Final advice

- Do not wait to “finish the theory” before building. Alternate a concept, a calculation, a code experiment, and a short write-up.
- For every project, compare against a simple baseline and report failures honestly.
- Your strongest AI-engineering advantage is the combination of ML understanding, reliable backend engineering, evaluation discipline, and production observability.